

Best Matching 25 (BM25) is a classic information retrieval algorithm based on TF-IDF. Given a query q that contains keywords (terms) $q_1, \dots, q_n$, the BM25 score on document d is

$$\text{BM25}_{q,d} = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f_{q_i,d} \cdot (k_1 + 1)}{f_{q_i,d} + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avg. } dl})},$$

where

$$\text{IDF}(q_i) = \log \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

using IDF with standard smoothing. $f_{q_i,d}$ is the term frequency of keyword q_i in document d , k_1 the control parameter for TF saturation, b the control parameter for document length dl , N the number of documents in the corpus, $n(q_i)$ the number of documents containing the keyword q_i .

The goal of this assignment is to implement top- k retrieval with BM25 from scratch: finding the most relevant top- k documents using a query of multiple keywords. The corpus you will be using is the Cranfield Collection, a foundational set of test collections developed for evaluating information retrieval systems. It has 1,400 documents, 225 queries, and 1,837 qrels.

You can call the Python API to download the corpus. First, install `ir_datasets` via your package manager such as `pip install ir_datasets`.

You may have a brief look at the corpus for `queries`, `docs`, `qrels`, and `metadata` after installation

```
import ir_datasets
dataset = ir_datasets.load("cranfield")
for query in dataset.queries_iter():
    print(query) # namedtuple<query_id, text>
    break

for doc in dataset.docs_iter():
    print(doc) # namedtuple<doc_id, title, text, author, bib>
    break

for qrel in dataset.qrels_iter():
    print(qrel) # namedtuple<query_id, doc_id, relevance, iteration>
    break
```

Follow the instruction in the `Assignment1.ipynb` and complete the implementation in the Jupyter Notebook.

Deadline: March 19, 23:59, CET.

Notice: You are only allowed to use built-in Python libraries such as numpy/pandas for the implementation, use of any full-packaged solutions are NOT permitted.

Task 1 BM25 Tokenizer

Implement a tokenizer that converts a text into a list of keywords: `tokenize(text) -> List[str]`.

Requirements:

- lowercase everything (e.g., UZH -> uzh, Machine Learning -> machine learning);
- split on alphanumeric words (e.g., a-z, 0-9);
- word stemming (e.g., working/worked/works -> work)
- drop empty tokens (e.g., retrieval -> retrieval);

- stopwords removal (e.g., the document has a title -> document title).

Notice: You may use NLTK for linguistic resources such as the stopwords list and word stemming (e.g., Porter Stemmer).

Task 2 BM25 Retriever

Implement the document retriever using BM25 that finds the best top- k documents for a given query.

Requirements:

- precompute N , dl , avg. dl , and $n(q_i)$;
- compute $f_{q_i,d}$ on the fly while through the corpus;
- calculate BM25 scores, rank them, and take the top- k .

Notice: for `relevance` attribute in `qrels`, you may treat -1 and 0 as non-relevant.

Task 3 BM25 Evaluator

Find the best set of hyperparameters k_1 and b by measuring the performance on Recall@10, Precision@10, MRR@10 and nDCG@10.

Requirements:

- evaluate the top- k retrieved documents for each metric (your code should allow specification of which metric to be used);
- perform grid search for $k_1 \in \{0.60, 1.20, 1.80\}$ and $b \in \{0.25, 0.50, 0.75\}$;
- report the best configuration by filling the table below;

	Recall@10	Precision@10	MRR@10	nDCG@10
$k_1 = 0.60, b = 0.25$				
$k_1 = 0.60, b = 0.50$				
$k_1 = 0.60, b = 0.75$				
$k_1 = 1.20, b = 0.25$				
$k_1 = 1.20, b = 0.50$				
$k_1 = 1.20, b = 0.75$				
$k_1 = 1.80, b = 0.25$				
$k_1 = 1.80, b = 0.50$				
$k_1 = 1.80, b = 0.75$				

- justify your choice of best configuration.

Task 4 Error Analysis

Pick two queries, one where BM25 retrieved great documents, one where BM25 failed the retrieval.

Requirements:

- show top 5 retrieved documents;
- explain why the ranking makes sense or not;
- propose one improvement.